Table 8-14. (*continued*)

`strtr()`	Replace single-byte characters with alternative byte values, or replace byte sequences with alternatives
`substr_replace()`	Replace specified bytes with a new byte sequence
`str_replace()`	Replace all occurrences of a sequence with a new sequence
`strrev()`	Reverse the byte order of a string
`trim()`	Strip specified bytes from the beginning or end of string
`ucfirst()`, `mb_ucfirst()` `ucwords()`	Make the first character of a string uppercase, or the first char of all words in a string
`wordwrap()`	Add a specified byte sequence every N bytes, with additional logic for long words
`mb_convert_case()`	Multi-byte safe case conversion: upper, lower, sentence, etc.
`mb_strimwidth()`	Truncate string to specified character width
`mb_substitute_character`	Get/set substitute character

Now, that's quite a number of functions to cover, and it will help if we categorize them further. I've grouped them into general functionalities: insertion, substitution, edge transformation (functions that affect the beginning or end of the string), and whole string transformation.

Insertion

These functions all share a common feature: they will add bytes to an existing string. A common reason for doing this is to prepare a long string of text for rendering in a fixed-width display. This might be a bash terminal or a web page (indeed, one of the functions will insert an HTML character sequence). Another scenario you're likely to encounter as a PHP developer is needing to render numeric strings in a regional format, which will require the insertion of spaces, commas, and so forth. We'll also look at the venerable `printf()` family of functions here, which is how string variable interpolation worked in the dark ages.